

CC1_IAA_L3 Informatique_2025/26 (séance de TP2, 02 février 2026)

À rendre : fichier(s) de code Python implémentant les fonctions et programmes demandés ci-après (tel qu'en fin de séance - 20% note finale).

Il s'agit d'implémenter un ensemble de fonctions et quelques programmes qui les agencent/testent, pour une prise en main de `pandas/sklearn/PIL` du projet d'apprentissage automatique. La librairie `PIL` est à privilégier pour les fonctions élémentaires sur les images.

Les images brutes sont dans le répertoire `Data`, réparties dans deux sous-répertoires : `Mer` contenant les images étiquetées « paysage avec mer/océan » (+1), et `Ailleurs` contenant les images étiquetées « sans paysage mer/océan » (-1).

- 1) **Explorer les données de ces répertoires** (nombres, tailles, formats, cohérence de l'étiquette, etc.).
- 2) **Programmer et tester les trois fonctions ci-après** (éventuellement en créant des fonctions intermédiaires) :

`buildSampleFromPath (path1, path2)` qui retourne un échantillon `s` de données issu des répertoires `path1` (les données +1, avec paysage maritime) et `path2` (les données -1, sans paysage maritime).

`resizeImage(i, h, l)` qui retourne l'image retaillée pour qu'elle soit de taille `h*l` (via paramètres `hauteur` et `largeur`).

`computeHisto(i)` qui calcule et retourne (et/ou stocke) l'histogramme de couleurs de l'image `i`, à l'aide de la librairie `PIL`.

Il est fortement conseillé (mais non imposé) de représenter l'échantillon `s` sous forme d'une liste (ou tableau) d'images, chaque image étant une entrée d'un dictionnaire :

```
Image= { name_path : « ... »,
         resized_image : ...,
         X_histo : ...,
         y_true_class : +1/-1,
         y_predicted_class : none/+1/-1
       }
```

Ce dictionnaire pourra évoluer au fil du projet selon les besoins.

- 3) **Programmer et tester les deux fonctions ci-après**, qui permettent (1) d'apprendre un modèle à partir de l'échantillon `s` via un algo (pour l'instant avec l'algorithme `NaiveBayes` vu en cours), puis (2) une fois ce modèle appris (via un `fit`), de calculer la prédiction sur une image. *En apprentissage comme en*

prédiction, on considère pour ce CC1 que les fonctions utilisent l'histogramme de couleurs de chaque image comme description de cette image (X_histo).

`fitFromHisto(S, algo)` qui calcule et retourne un modèle, `model`, appris de `S` (où chaque image est représentée par un histogramme de couleurs), avec l'algorithme `algo`. Dans un premier temps, l'algorithme est `NaiveBayes`, donc le modèle retourné sera l'instance de `NaiveBayes` une fois fittée sur les données `S` ; mais à terme il est recommandé qu'un `algo` soit une entrée de dictionnaire :

```
algo= {name : « ... », (le nom de la classe sklearn)
      hyper_param : dictionnaire d'hyperparamètres
      }
```

`predictFromHisto(S, model)` qui calcule la prédiction du modèle `model` (préalablement appris) sur les données de l'échantillon `S`, chaque donnée (image) de `S` représentée par son histogramme de couleurs. La fonction retourne la liste des prédictions, et/ou modifie l'échantillon s'il est une liste d'entrées de dictionnaire comme suggéré ci-avant (affectation du champ `y_predicted_class`). Cette fonction peut faire appel à une autre fonction qui calcule la prédiction, pour une et une seule image.

- 4) **Programmer la fonction qui permet de calculer l'erreur empirique du modèle appris sur les histogrammes.** On rappelle que l'erreur empirique est la proportion d'erreurs commises par le modèle sur les données de l'échantillon qui a permis d'apprendre ce modèle.
- 5) **Programmer une fonction qui permet d'estimer – par cross-validation – l'erreur réelle d'un modèle appris** avec l'algorithme `NaiveBayes` sur les histogrammes de couleurs des images.

Pour les groupes rapides (qui ont fini avant la fin du TP), rajouter les fonctions nécessaires pour les mêmes traitements mais où chaque image est représentée par `X_pixelbw` (tableau de pixels niveaux de gris).